

Compiler Design - CST302

Module 3: Advanced Topics - CLR(1) and LALR(1) Parsing

CLR(1) Canonical LR Parsing | LR(1) Items | Closure and GOTO | Canonical Collection | CLR Table | LALR(1) Merging | Parser Comparison

1. Canonical LR / CLR(1) Parsing

CLR(1) is the **most powerful LR parser**. It uses **lookahead symbols embedded directly in LR items** to make precise reduce decisions, eliminating many conflicts that SLR cannot resolve.

CLR recognises the **largest class of grammars** among all LR parsers.

2. Why CLR is Needed - Problem with SLR

SLR decides REDUCE based on **FOLLOW(A)** - the set of all terminals that can follow A anywhere in the grammar. This is too general: a terminal in FOLLOW(A) may not actually appear after A in the current parsing context, causing **false reductions / conflicts**.

Parser	Reduce Decision Based On	Problem
SLR(1)	FOLLOW(A) - global lookahead	May reduce when it should not -> conflicts
CLR(1)	Exact lookahead stored in each LR(1) item	No false reductions; more precise

3. LR(1) Items

An **LR(1) item** = LR(0) item + one lookahead symbol.

Format:

```
[ A -> alpha . beta , a ]
```

Part	Symbol	Meaning
Production	A -> alpha beta	The grammar rule being tracked
Dot position	.	How much of RHS has been parsed
Lookahead	a	Terminal that must follow after A is reduced

Key rule: reduce using A -> alpha ONLY when the next input symbol **equals the lookahead a**.

Examples of LR(1) items:

```
[ S' -> . S , $ ] -- nothing parsed yet; lookahead = $ [ C -> . cC , c/d ] --  
expecting c; lookahead = c or d [ C -> d . , c/d ] -- dot at end -> REDUCE by C->d  
when input is c or d [ C -> d . , $ ] -- REDUCE by C->d only when input is $
```

4. Closure of LR(1) Items

The closure rule for LR(1) is like LR(0) but **propagates lookaheads**.

Closure Rule:

If $[A \rightarrow \alpha \cdot B \beta, a]$ is in the set, then for every production $B \rightarrow \gamma$, add: $[B \rightarrow \cdot \gamma, b]$ for each b in $\text{FIRST}(\beta a)$

$\text{FIRST}(\beta a)$: compute FIRST of string β followed by terminal a . If β can derive empty string, then a is also included in the lookahead.

Step-by-step closure of $\{[S' \rightarrow \cdot S, \$]\}$ for grammar $S \rightarrow CC, C \rightarrow cC|d$:

Item Added	Triggered By	Lookahead = $\text{FIRST}(\dots)$
$S' \rightarrow \cdot S, \$$	Initial item	--
$S \rightarrow \cdot CC, \$$	$S' \rightarrow \cdot S, \$$ (S is after dot)	$\text{FIRST}(\text{empty } \$) = \{\$ \}$
$C \rightarrow \cdot cC, c/d$	$S \rightarrow \cdot CC, \$$ (C after dot; $\beta=C$)	$\text{FIRST}(C\$) = \{c,d\}$
$C \rightarrow \cdot d, c/d$	$S \rightarrow \cdot CC, \$$ (C after dot; $\beta=C$)	$\text{FIRST}(C\$) = \{c,d\}$

5. GOTO Operation for LR(1)

$\text{GOTO}(I, X)$ collects all items in I where the dot is immediately before X , moves the dot past X , and takes the closure of the result. **Lookaheads are carried over unchanged.**

If $[A \rightarrow \alpha \cdot X \beta, a]$ is in I , then $\text{GOTO}(I, X)$ contains $\text{closure}(\{[A \rightarrow \alpha X \cdot \beta, a]\})$

6. Augmented Grammar for CLR Example

Original grammar: $S \rightarrow CC, C \rightarrow cC | d$

Production No.	Production	Notes
0	$S' \rightarrow S$	New start production (augmented)
1	$S \rightarrow CC$	
2	$C \rightarrow cC$	
3	$C \rightarrow d$	

7. Canonical Collection of LR(1) Items - All 10 States

The DFA for this grammar has **10 states (I0 to I9)**. Each state is a set of LR(1) items derived by closure and GOTO operations.

I0 (Start State)

LR(1) Items	Transitions / Action
$S' \rightarrow \cdot S, \$$ $S \rightarrow \cdot CC, \$$ $C \rightarrow \cdot cC, c/d$ $C \rightarrow \cdot d, c/d$	$\text{GOTO}(I0, S)=I1$ $\text{GOTO}(I0, C)=I2$ $\text{GOTO}(I0, c)=I3$ $\text{GOTO}(I0, d)=I4$

I1 = $\text{GOTO}(I0, S)$

LR(1) Items	Transitions / Action
-------------	----------------------

S' -> S . , \$	ACCEPT state
I2 = GOTO(I0, C)	
LR(1) Items	Transitions / Action
S -> C . C , \$ C -> . cC , \$ C -> . d , \$	GOTO(I2,C)=I5 GOTO(I2,c)=I6 GOTO(I2,d)=I7
I3 = GOTO(I0, c)	
LR(1) Items	Transitions / Action
C -> c . C , c/d C -> . cC , c/d C -> . d , c/d	GOTO(I3,C)=I8 GOTO(I3,c)=I3 GOTO(I3,d)=I4
I4 = GOTO(I0, d)	
LR(1) Items	Transitions / Action
C -> d . , c/d	REDUCE by C->d when input is c or d
I5 = GOTO(I2, C)	
LR(1) Items	Transitions / Action
S -> CC . , \$	REDUCE by S->CC when input is \$
I6 = GOTO(I2, c)	
LR(1) Items	Transitions / Action
C -> c . C , \$ C -> . cC , \$ C -> . d , \$	GOTO(I6,C)=I9 GOTO(I6,c)=I6 GOTO(I6,d)=I7
I7 = GOTO(I2, d)	
LR(1) Items	Transitions / Action
C -> d . , \$	REDUCE by C->d when input is \$
I8 = GOTO(I3, C)	
LR(1) Items	Transitions / Action
C -> cC . , c/d	REDUCE by C->cC when input is c or d
I9 = GOTO(I6, C)	
LR(1) Items	Transitions / Action
C -> cC . , \$	REDUCE by C->cC when input is \$

8. CLR(1) Parsing Table

Built from the 10-state DFA. ACTION uses exact lookaheads from LR(1) items.

Rule	Condition	Table Entry
Shift	[A -> alpha . a beta , b] in li and GOTO(li,a)=lj	ACTION[i, a] = shift j
Reduce	[A -> alpha . , a] in li (dot at end)	ACTION[i, a] = reduce A->alpha
Accept	[S' -> S . , \$] in li	ACTION[i, \$] = accept
GOTO	GOTO(li, A) = lj (A is non-terminal)	GOTO[i, A] = j

CLR(1) Parsing Table for S->CC, C->cC|d:

State	c (ACTION)	d (ACTION)	\$ (ACTION)	S (GOTO)	C (GOTO)
0	S3	S4	--	1	2
1	--	--	Accept	--	--
2	S6	S7	--	--	5
3	S3	S4	--	--	8
4	R3	R3	--	--	--
5	--	--	R1	--	--
6	S6	S7	--	--	9
7	--	--	R3	--	--
8	R2	R2	--	--	--
9	--	--	R2	--	--

R1 = reduce S->CC | R2 = reduce C->cC | R3 = reduce C->d | S3/S4/S6/S7 = shift to that state | Accept at state 1 on \$

9. LALR(1) Parsing

LALR = LookAhead LR. It is the **most widely used LR parser** in practice (used by YACC/Bison). LALR achieves a balance: more powerful than SLR and uses far less memory than CLR.

Property	LALR(1)	Compared To
Number of States	Same as SLR	Much fewer than CLR
Parsing Power	More than SLR	Slightly less than CLR
Table Size	Medium (same as SLR)	CLR is much larger
Practical Use	YACC, Bison, most compilers	CLR rarely used in practice

10. Core of an LR(1) Item

The **core** of an LR(1) item is the LR(0) part -- i.e., the item **without the lookahead**.

LR(1) item: [A -> alpha . beta , a] Core: A -> alpha . beta (lookahead ignored)

Two states have the **same core** if they contain the same set of LR(0) items (possibly with different lookaheads). These states can be **merged in LALR**.

11. Merging States in LALR

States with identical cores are merged. Their lookahead sets are **unioned together**.

Example - from the 10-state CLR DFA:

CLR State	Items	Core (LR(0) part)
I4	C -> d . , c/d	C -> d .
I7	C -> d . , \$	C -> d .
Merged I47	C -> d . , c/d/\$	C -> d . (same core -> MERGE)

All mergeable state pairs for this grammar:

CLR States to Merge	Core	Merged State	Combined Lookaheads
I3 + I6	C -> c.C, C->.cC, C->.d	I36	c/d and \$
I4 + I7	C -> d .	I47	c/d/\$
I8 + I9	C -> cC .	I89	c/d/\$

12. LALR(1) Construction Steps

Step	Action
1	Write the Augmented Grammar
2	Build the full CLR(1) canonical collection (all LR(1) item sets)
3	Identify all pairs of states that have the same LR(0) core
4	Merge those states -- union their lookahead sets
5	Update all GOTO transitions to point to merged states
6	Build ACTION and GOTO tables from the merged states
7	Check for conflicts in merged table (rare but possible)

If a conflict arises after merging, the grammar is CLR(1) but not LALR(1). Such conflicts are called **LALR merge conflicts**.

13. SLR vs LALR vs CLR - Full Comparison

Feature	SLR(1)	LALR(1)	CLR(1)
Lookahead source	FOLLOW(A)	Merged LR(1) lookaheads	Exact LR(1) items

No. of states	Fewest	Same as SLR	Most (can be 10x SLR)
Table size	Small	Medium (= SLR)	Very Large
Grammar class	Smallest	Intermediate	Largest
Conflicts	More	Fewer than SLR	Fewest
Practical use	Teaching/academic	YACC, Bison, compilers	Theoretical
Memory cost	Low	Low	High

14. Complete Advanced Concept Flow - Module 3

Topic	Key Concept	Exam Tip	Weight
LR(1) Items	LR(0) item + lookahead symbol	Write items with [A->alpha.beta, a] notation	High
Closure (LR(1))	Add [B->.gamma, FIRST(beta a)] for each B->.gamma	Lookahead = FIRST of remaining string + epsilon	Very High
GOTO (LR(1))	Move dot across symbol; carry lookahead over	Same as LR(0) but include lookahead in result	High
CLR States I0-I9	10 states for S->CC grammar; build DFA	Know all 10 states for S->CC, C->cC d	Very High
CLR Table	Reduce only on exact lookahead in item	Fill ACTION using lookaheads, not FOLLOW	Very High
LALR Core	Core = LR(0) part without lookahead	States with same core -> candidates to merge	High
LALR Merging	Union lookaheads of states with identical cores	Know merge pairs: I4+I7, I3+I6, I8+I9	Very High
LALR Table	Build from merged states; fewer rows than CLR	Same structure as CLR but fewer states	Very High